



ENGINEERING BLOG

Menu

BLOG | DATA SCIENCE & ENGINEERING

Monte Carlo Simulations: Separating Signal from Noise in Sampled Success Metrics

Check out this guide for using a Monte Carlo simulation to identify the size and confidence percentage of your sampled success metric.

Published on Jan 3, 2023



Usually, when you set success metrics you're able to directly measure the value of interest in its entirety. For example, Shopify can measure Gross Merchandise Volume (GMV) with precision because we can query our databases for every order we process. However, sometimes the information that tells you whether you're having an impact isn't available, or is too expensive or time consuming to collect. In these cases, you'll need to rely on a sampled success metric.

In a one-shot experiment, you can estimate the sample size you'll need to achieve a given confidence interval. However, success metrics are generally tracked over time, and you'll want to evaluate each data point in the context of the trend, not in isolation. Our confidence



ENGINEERING BLOG

Menu

and share how to implement this in Python and pandas so you can do it yourself.

What is a Monte Carlo Simulation?

A Monte Carlo simulation can be used to generate a bunch of random inputs based on real world assumptions. It does this by feeding these inputs through a function that approximates the real world situation of interest, and observing the attributes of the output to understand the likelihood of possible outcomes given reasonable scenarios.

In the context of a sampled success metric, you can use the simulation to understand the tradeoff between:

- Your sample size
- Your ability to extract trends in the underlying population metric from random noise

These results can then be used to explain complex statistical concepts to your non-technical stakeholders. How? You'll be able to simply explain the percentage of certainty your sample size yields, against the cost of collecting more data.

Using a Monte Carlo Simulation to Estimate Metric Variability

To show you how to use a Monte Carlo simulation for a sampled success metric, we'll turn to the [Shopify App Store](#) as an example. The Shopify App Store is a marketplace where our merchants can find apps and plugins to customize their store. We have over 8,000 apps solving a range of problems. We set a high standard for app quality, with over 200 minimum requirements focused on security, functionality, and ease of use. Each app needs to meet these requirements in order to be listed, and we have various manual and automated app review processes to ensure these requirements are met.

We want to continuously evaluate how our review processes are improving the quality of our app store. At the highest level, the question we want to answer is, "How good are our apps?". This can be represented quantitatively as, "How many requirements does the



ENGINEERING BLOG

Menu

whether our various app review processes are having an impact on improving the quality of our apps. This is our sampled success metric.

With mock data and parameters, we'll show you how we can use a Monte Carlo simulation to identify how many apps we need to audit each month to have confidence in our sampled success metric. We'll then repeatedly simulate auditing randomly selected apps, varying the following parameters:

- Sample size
- Underlying trend in issue rate

To understand the sensitivity of our success metric to relevant parameters, we need to conduct five steps:

1. Establish our simulation metrics
2. Define the distribution we're going to draw our issue count from
3. Run a simulation for a single set of parameters
4. Run multiple simulations for a single set of parameters
5. Run multiple simulations across multiple parameters

To use a Monte Carlo simulation, you'll need to have a success metric in mind already. While it's ideal if you have some idea of its current value and the distribution it's drawn from, the whole point of the method is to see what range of outcomes emerges from different plausible scenarios. So, don't worry if you don't have any initial samples to start with.

Step 1: Establishing Our Simulation Metrics

We start by establishing simulation metrics. These are different from our success metric as they describe the *variability* of our sampled success metric. Metrics on metrics!

For our example, we'll want to check on this metric on a monthly basis to understand whether our approach is working. So, to establish our simulation metric, we ask ourselves,



ENGINEERING BLOG

Menu

simulate these metrics for one year into the future.

On top of our simulation metric, we'll also want to measure the mean absolute percentage error (MAPE). MAPE will help us identify the percentage by which the shop issue rate departs from the true underlying distribution each month.

Now, with our simulation metrics established, we need to define what distribution we're going to be pulling from.

Step 2: Defining Our Sampling Distribution

For the purpose of our example, let's say we're going to generate a year's worth of random app audits, assuming a given monthly decrease in the population shop issue rate (our success metric). We'll want to compare the sampled shop issue rate that our Monte Carlo simulation generates to that of the population that generated it.

We generate our Monte Carlo inputs by drawing from a random distribution. For our example, we've identified that the number of issues an app has is well represented by the Poisson distribution which models the sum of a collection of independent Bernoulli trials (where the evaluation of each requirement can be considered as an individual trial). However, your measure of interest might match another, like the normal distribution. You can find more information about fitting the right distribution to your data [here](#).

The Poisson distribution has only one parameter, λ (lambda), which ends up being both the mean and the variance of the population. For a normal distribution, you'll need to specify both the population mean and the variance.

Hopefully you already have some sample data you can use to estimate these parameters. If not, the code we'll work through below will allow you to test what happens under different assumptions.

Step 3: Running Our Simulation with One Set of Parameter Values



ENGINEERING BLOG

Menu

percentage by which we're going to decrease the λ (or mean issue count) each month.

Finally, we need to set how many random audits we're going to conduct (aka our sample size). As the sample size goes up, so does the cost of collection. This is a really important number for stakeholders. We can use our results to help communicate the tradeoff between certainty of the metric versus the cost of collecting the data.

Now, we're going to write the building block function that generates a realistic sampled time series given some assumptions about the parameters of the distribution of app issues. For example, we might start with the following assumptions:

1. Our population mean is 10 issues per install. This is our λ parameter.
2. Our shop issue rate decreases 5 percent per month. This is how much of an impact we expect our app review processes to have.

Note that these assumptions could be wrong, but the goal is not to get your assumptions right. We're going to try lots of combinations of assumptions in order to understand how our simulation metrics respond across reasonable ranges of input parameters.

For our first simulation, we'll start with a function that generates a time series of issue counts, drawn from a distribution of apps where the population issue rate is in fact decreasing by a given percentage per month. For this simulation, we'll draw from 100 sample time series. This sample size will provide us with a fairly stable estimate of our simulation metrics, without taking too long to run. Below is the output of the simulation:



ENGINEERING BLOG

Menu

```

1  """ Given population mean (l) issue count, draw a random issue count
2  from the relevant Poisson distribution"""
3
4
5
6
7
8  return np.random.poisson(l)
9
10
11
12 def generate_time_series(periods=12,
13     monthly_decrease=0.05,
14     starting_mean=5,
15     audits_per_period=100):
16     """
17     Generate a time series of sampled apps with
18     randomly generated issue counts.
19
20     Also returns the true population SIR over the period.
21     """
22
23     cols = range(periods)
24     index = range(audits_per_period)
25     df = pd.DataFrame(index=index, columns=cols)
26     sample_df = pd.DataFrame(index=range(audits_per_period), columns=cols)
27
28     mean = starting_mean
29
30     true_sir = []
31     for period in cols:
32         app_error_counts = [generate_error_count(mean) for x in range(audits_per_period)]
33         df[period] = app_error_counts
34         multiplier = (1 - monthly_decrease)
35         true_sir.append(mean)
36
37         mean = mean * multiplier
38
39     return (np.array(true_sir), df)
40
41 true_sir, sample_df = generate_time_series(audits_per_period=100, monthly_decrease=0.05, periods=12)
42
43 # Plot the generated data against the mean of the distribution that generated it
44 fig, ax = plt.subplots()
45 sample_df.mean().rename("Sampled Mean").plot(ax=ax, color='black', linestyle='--')

```



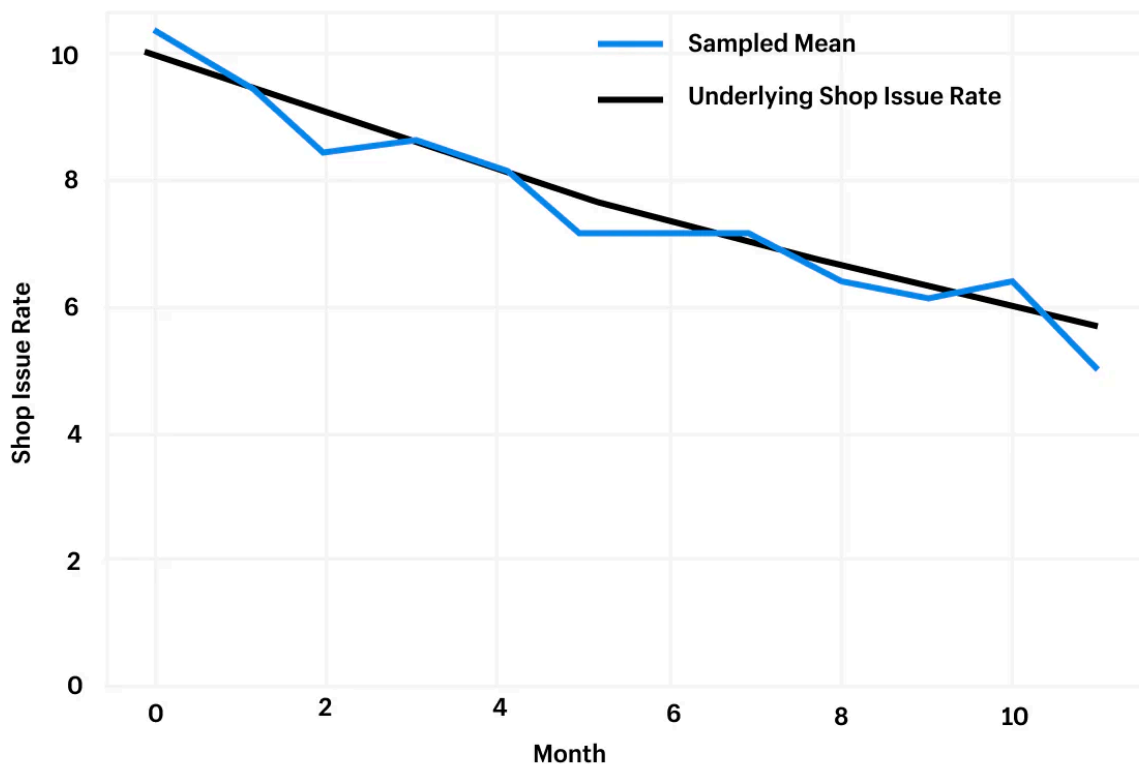
ENGINEERING BLOG

Menu

`simulate_shop_issue_rate_time_series.py` hosted with ❤ by GitHub

[view raw](#)

This function returns a sample dataset of $n=\text{audits_per_period}$ apps over $m=\text{periods}$ months, where the number of issues for each app is drawn from a Poisson distribution. In the chart below, you can see how the sampled shop issue rate varies around the true underlying number. We can see 10 mean issues decreasing 5 percent every month.



Our first Monte Carlo simulation with one set of parameter values

Now that we've run our first simulation, we can calculate our variability metrics MAPE and 1mDO. The below code block will calculate our variability metrics for us:



ENGINEERING BLOG

Menu

```

7     error = true_sir - sample_sir
8     abs_error = error.abs()
9     mape = (abs_error/true_sir).mean()
10
11     decreases_observed_over_period = []
12     for period in sample_sir.index:
13         diffs = sample_sir.diff(period+1)
14         n_diffs = diffs.notna().sum()
15         decreases_observed = (diffs < 0).sum() / n_diffs
16         decreases_observed_over_period.append(decreases_observed)
17
18     return mape, np.array(decreases_observed_over_period)
19
20 get_summary_stats(true_sir, sample_df)

```

summarize_time_series_variability.py hosted with ❤ by GitHub

[view raw](#)

This code will tell us how many months it will take before we actually see a decrease in our shop issue rate. Interpreted another way, "How long do we need to wait to act on this data?".

In this first simulation, we found that the MAPE was 4.3 percent. In other words, the simulated shop issue rate differed from the population mean by 4.3 percent on average. Our 1MDO was 72 percent, meaning our sampled metric decreased in 72 percent of months. These results aren't great, but was it a fluke? We'll want to run a few more simulations to identify confidence in your simulation metrics.

Step 4: Running Multiple Simulations with the Same Parameter Values

The code below runs our `generate_time_series` function `n=iterations` times with the given parameters, and returns a DataFrame of our simulation metrics for *each* iteration. So, if we run this with 50 iterations, we'll get back 50 time series, each with 100 sampled audits per month. By averaging across iterations, we can find the averages of our simulation metrics.



ENGINEERING BLOG

Menu

```

1         starting_mean=7.0,
8         audits_per_period=100):
9
10        #results = pd.DataFrame(index=range(iterations), columns=['MAE', '1p decreases observed', '1p
11        rows = []
12        sample_dfs = []
13        for i in range(iterations):
14            true_sir, sample_df = generate_time_series(periods=periods,
15                                                    monthly_decrease=monthly_decrease,
16                                                    starting_mean=starting_mean,
17                                                    audits_per_period=audits_per_period)
18            sample_dfs.append(sample_df)
19            mae, dobs = get_summary_stats(true_sir, sample_df)
20            rows.append([mae, dobs[0], dobs[1], dobs[2]])
21
22        return pd.DataFrame(rows, columns=["MAPE", "1MDO", "2MDO", "1QDO"]), sample_dfs, t
23    results_df, sample_dfs, true_sir = run_simulation(200)
24    results_df.mean()

```

run_simulation_multiple_times.py hosted with ❤ by GitHub

[view raw](#)

Now, the number of simulations to run depends on your use case, but 50 is a good place to start. If you're simulating a manufacturing process where millimeter precision is important, you'll want to run hundreds or thousands of iterations. These iterations are cheap to run, so increasing the iteration count to improve your precision just means they'll take a little while longer to complete.

Four sample Monte Carlo simulations with the same parameter values

For our example, 50 sampled time series enables us with enough confidence that these metrics represent the true variability of the shop issue rate. That is, as long as our real world inputs are within the range of our assumptions.



ENGINEERING BLOG

Menu

analogous to the grid search approach to hyperparameter tuning in machine learning. Remember, for our app store example, we want to identify the impact of our review processes on the metric for both the monthly percentage decrease and monthly sample size.

We'll use the code below to specify a reasonable range of values for the monthly impact on our success metric, and some possible sample sizes. We'll then run the `run_simulation` function across those ranges. This code is designed to allow us to search across any dimension. For example, we could replace the monthly decrease parameter with the initial mean issue count. This allows us to understand the sensitivity of our metrics across more than two dimensions.



ENGINEERING BLOG

Menu

```

/
8  ITERATIONS=100
9
10 def grid_results(iterations, row_param, row_param_values, col_param, col_param_values, **kv
11     metric_names = ['MAPE','1MDO','2MDO','1QDO']
12
13     result_columns = pd.MultiIndex.from_product([col_param_values, metric_names])
14     result_columns.name = col_param
15
16     results = pd.DataFrame(columns=result_columns, index=row_param_values)
17     for col_val in col_param_values:
18         row = []
19         for row_val in row_param_values:
20             params = {row_param: row_val, col_param: col_val}
21             metrics, _, _ = run_simulation(iterations, **params, **kwargs)
22             mean_metrics = metrics.mean()
23
24             for metric in metric_names:
25                 results.at[row_val, (col_val, metric)] = mean_metrics.loc[metric]
26     return results
27
28 results = grid_results(ITERATIONS,
29     'monthly_decrease',
30     DECREASES,
31     'audits_per_period',
32     AUDIT_COUNTS,
33     starting_mean=STARTING_MEAN)

```

grid_search.py hosted with ❤ by GitHub

[view raw](#)

The simulation will produce a range of outcomes. Looking at our results below, we can tell our stakeholders that if we start at 10 average issues per audit, run 100 random audits per month, and decrease the underlying issue rate by 5 percent each month, we should see monthly decreases in our success metric 83 percent of the time. Over two months, we can expect to see a decrease 97 percent of the time.



ENGINEERING BLOG

Menu

stakeholders can understand and implement. For example, we can look to our results and communicate that an additional 50 audits per month would yield quantifiable improvements in certainty. This insight can enable our stakeholders to make an informed decision about whether that certainty is worth the additional expense.

And there we have it! The next time you're looking to separate signal from noise in your sampled success metric, try using a Monte Carlo simulation. This fundamental guide just scratches the surface of this complex problem, but it's a great starting point and I hope you turn to it in the future.

Tom is a data scientist working on systems to improve app quality at Shopify. In his career, he tried product management, operations and sales before figuring out that SQL is his love language. He lives in Brooklyn with his wife and enjoys running, cycling and writing code.

Are you passionate about solving data problems and eager to learn more about Shopify? Check out openings on our [careers page](#).

by Tom Lowe

Published on Jan 3, 2023



SHARE ARTICLE





ENGINEERING BLOG

Menu

DEVELOPMENT

Introducing Ruvy

2023-10-18

DEVELOPER TOOLING

Building a ShopifyQL Code Editor

2023-09-11

APPS

Shopify's r
platform

2023-07-26



WORK FROM ANYWHERE WITH SHOPIFY

See our open roles and learn more about our digital by design culture.

[See open roles](#)

Shopify

Support

Developers



ENGINEERING BLOG

Menu

Press and Media

Shopify Academy

Partners

Shopify Community

Affiliates

Legal

Service status

Products

Global Impact

Solutions

Shop

Sustainability

Online Store Builder

Shop Pay

Build Black

Website Builder

Shopify for Enterprise

Accessibility

Ecommerce Website

Research

Terms of service

Privacy policy

Sitemap

Your Privacy Choices